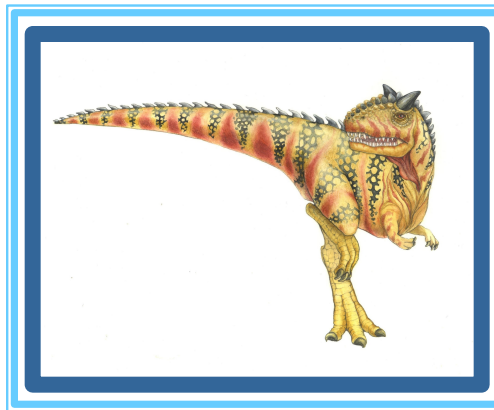


Process Management

Rakhshan Imtiaz





Operations on Processes





Process Creation

- Parent process create children processes, which, in turn create other processes, forming a tree of processes
- Resource sharing
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution
 - Parent and children execute concurrently
 - Parent waits until children terminate





init process

- **init** is parent of all Linux processes with PID or process ID of 1.
- init stands for initialization.
- It is the first process to start when a computer boots up and runs until the system shuts down





Process Creation

- These new processes may themselves go on to create new processes.
- All of the processes in the system are descended from the init kernel thread.
- You can see the family relationship between the running processes in a Linux system using the **ps** command
- A new process is created by a **fork()** system call





The fork() system call

- A new process known as a "child process" is created with the fork system call which runs concurrently with the process called the parent process.
- The existing process is called the parent process.





The fork() system call

- fork system call in OS returns an integer value and requires no arguments. After the creation of a new child process, both processes then execute the next command following the fork system call.
- Therefore, we must separate the parent from the child by checking the returned value of the fork():





The fork() system call

- **Negative:** A child process could not be successfully created if the fork() returns a **negative** value.
- **Zero:** A new child process is successfully created if the fork() returns a **zero**.
- **Positive:** The positive value is the process ID of a child's process to the parent.





The fork() system call :

Example 1

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

//main function begins
int main()
{
    fork();
    print("the process is created using fork() system call\n");
    return 0;
}

// Using fork() system call single time
```





The fork() system call :

Output:

```
the process is created using fork() system call  
the process is created using fork() system call
```





fork() is called multiple times:Example-2

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

//main function begins
int main()
{
    fork();
    fork();
    fork();
    printf("this process is created by fork() system call");
    return 0;
}

// fork() is used 3 times:
```





fork() is called multiple times: Example-2

```
// Since the commands will be executed 2n times  
//  $2^3 = 8$   
this process is created by fork() system call  
this process is created by fork() system call  
this process is created by fork() system call  
this process is created by fork() system call  
this process is created by fork() system call  
this process is created by fork() system call  
this process is created by fork() system call  
this process is created by fork() system call
```





Example-3

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

void child() {
    printf("Child process is created by fork() system call");
}

void parent() {
    printf("parent process is created by fork() system call");
}

//main function begins
int main(void) {
    pid_t p = fork();

    if(p == 0) {
```

```
        child();
        exit(EXIT_SUCCESS);
    }
    else if(p > 0) {
        wait(NULL);
        parent();
    }
    else {
        printf("fork() system call is unable to create child process.");
    }

    return EXIT_SUCCESS;
}
```





Example-3

Child process is created by `fork()` system call
parent process is created by `fork()` system call





The wait() System Call

- The parent process will often want to wait until all child processes have been completed. This can be implemented with the wait() function call.
- **wait():** Blocks calling process until the child process terminates. If child process has already terminated, the wait() call returns immediately.





wait() system call-Example

```
#include<sys/types.h>
#include<stdio.h>
#include<sys/wait.h>
int main(){
    pid_t p;
    printf("before fork\n");
    p=fork();
    if(p==0)
        //child
        {
            printf("I am child having id %d\n",getpid());
            printf("My parent's id is %d\n", getppid());
        }
    else
        //parent
        {
            wait(NULL);
            printf("My child's id is %d\n",p);
            printf("I am parent having id %d\n",getpid());
        }
    printf("Common\n");
}
```





The wait() System Call

```
#include<unistd.h>
#include<sys/types.h>
#include<stdio.h>
#include<sys/wait.h>
int main()
{
    pid_t p;
    printf("before fork\n");
    p=fork();
    if(p==0)//child
    {
        printf("I am child having id %d\n",getpid());
        printf("My parent's id is %d\n",getppid());
    }
    else//parent
    {
        wait(NULL);
        printf("My child's id is %d\n",p);
        printf("I am parent having id %d\n",getpid());
    }
    printf("Common\n");
}
```





The wait() System Call

Output :

```
before fork
I am child having id 8039
My parent's id is 8038
Common
My child's id is 8039
I am parent having id 8038
Common
```





Sleep () System Call

```
// C program to demonstrate
// use of sleep function
// till 10 seconds in Linux.
#include <stdio.h>
#include <unistd.h>

int main()
{
    // This line will be executed first
    printf("Program to sleep for 10 second in Linux.\n");

    sleep(10);
    // after 10 seconds this next line will be executed.

    printf("This line will be executed after 10 second.");

    return 0;
}
```





Sleep () System Call

Output:

Program to sleep for 10 second in Linux.
This line will be executed after 10 second.





Zombie Process

- A zombie process refers to a process that has completed its execution and waiting for its parent to collect its exit status.
- A process which has finished the execution but still has entry in the process table to report to its parent process is known as a zombie process.
- A child process always first becomes a zombie before being removed from the process table.





Zombie Process

```
// A C program to demonstrate Zombie Process.  
// Child becomes Zombie as parent is sleeping  
// when child process exits.  
#include <stdlib.h>  
#include <sys/types.h>  
#include <unistd.h>  
int main() {  
    // Fork returns process id  
    // in parent process  
    int pid= fork();  
    // Parent process  
    if (pid > 0)  
        sleep(50);  
    // Child process  
    else  
        exit(0);  
    return 0;  
}
```





Orphan Process

- A process whose parent process no more exists i.e. either finished or terminated without waiting for its child process to terminate is called an orphan process.
- In the following code, parent finishes execution and exits while the child process is still executing and is called an orphan process now.
- However, the orphan process is soon adopted by init process, once its parent process dies.





Orphan Process

```
#include <unistd.h>
#include <stdlib.h>
int main() {
    int pid = fork();
    if (pid > 0) {
        // parent process code
        exit(0);
        // Parent terminates before the child process
    }
    if(pid == 0) {
        // child process code
        sleep(60);
    }
    return 0;
}
```





Orphan Process

```
// A C program to demonstrate Orphan Process.
// Parent process finishes execution while the child process is running.
//The child process becomes orphan.
#include<stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main() {
    // Create a child process
    int pid = fork();
    if (pid > 0)
        printf("in parent process");
        // Note that pid is 0 in child process
        // and negative if fork() fails
    else if (pid == 0)
    {
        sleep(60);
        printf("in child process");
    }
    return 0;
}
```





exit()

void exit(int *status*);

- After the program finishes execution, it calls *exit()*
- This system call:
 - takes the “result” of the program as an argument
 - closes all open files, connections, etc.
 - deallocates memory
 - deallocates most of the OS structures supporting the process





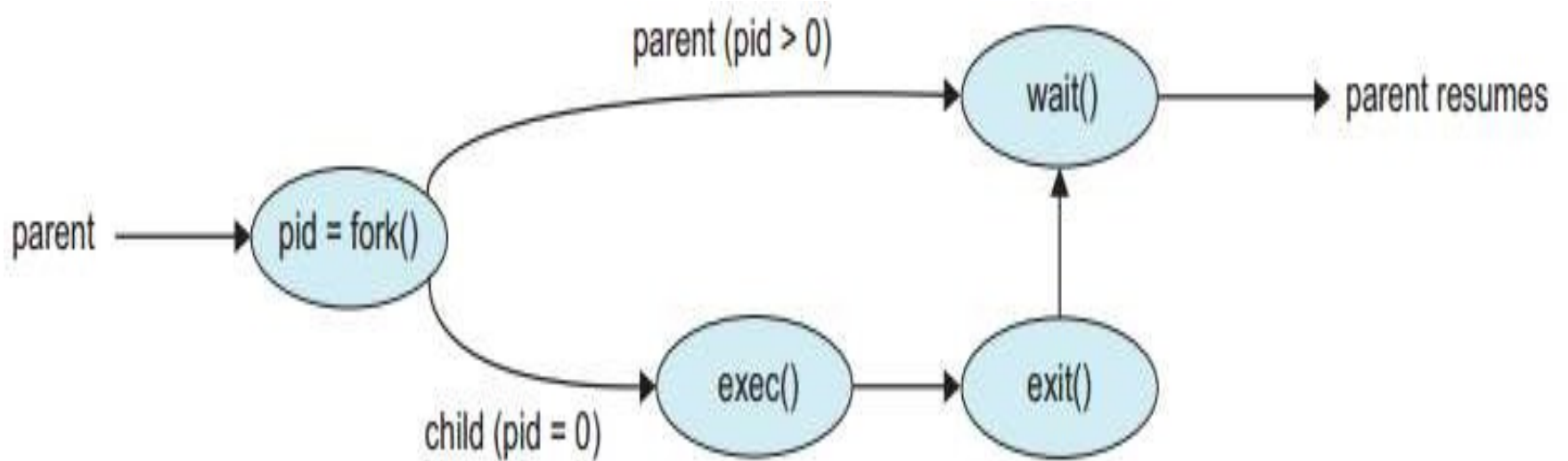
exec()

- We usually want the child process to run some other executable, so we use exec() system call after the fork system call.
- No new process create
- Process ID remains same
- Inherited process data and code is gone.
- Cannot return to the caller.





Process Creation



END

